Astana IT University · Computer Vision · Assignment 2

Level-Up Clothing Classification: ConvNeXt, SD-Turbo Synthetic Data and a Grad-CAM Demo

Student: [Full name], group [Group] · Instructor: Nurym Kaiyrkhan

GitHub: https://github.com/AlleSH419/clothing-classification-A2

Colab notebook + app: Open in Google Colab

Demo video: [insert video link]

Abstract

We upgrade an Assignment 1-style clothing classifier (5 classes: bottomwear, footwear, headwear, topwear, underwear) with two state-of-the-art components: an ImageNet-pretrained **ConvNeXt-Tiny** backbone and **SD-Turbo** text-to-image synthetic data for the weakest classes. On a held-out test set of 225 real images, ConvNeXt raises accuracy from **80.9 % to 98.2 %** (macro-F1 0.802 → 0.982). Adding 200 synthetic images raises it further to **98.7 %** (macro-F1 0.987), but slightly hurts the from-scratch CNN. We ship an interactive Gradio app (upload or camera → prediction → Grad-CAM → latency, 10–16 ms on a T4). Grad-CAM and a real-world phone test reveal a dataset shortcut that the test metric hides.

1. Introduction

Automatic garment categorization underpins e-commerce search, catalog tagging and recommendation. In Assignment 1 the task was solved with classical baselines. Here we keep the same topic and ask three questions. (1) How much does a modern pretrained backbone help over a CNN trained from scratch? (2) Can a one-step diffusion generator fill weak classes with useful synthetic data? (3) What does the model actually look at, and does its accuracy transfer to photos taken outside the catalog?

2. Dataset Update

Source. Fashion Product Images (Small), a public e-commerce catalog of 44,072 product photos (Hugging Face mirror ashraq/fashion-product-images-small). The dataset field subCategory is mapped to our classes: Shoes → footwear, Innerwear → underwear, Topwear → topwear, Headwear → headwear, Bottomwear → bottomwear.

Size, balance and split. 300 random images per class (headwear has only 293), 1,493 real images in total, split 70 / 15 / 15 per class with a fixed seed.

Augmentation (train only). Resize to 224×224, random horizontal flip, random rotation ±15°, color jitter (brightness, contrast and saturation 0.3), ImageNet normalization. Validation and test images are only resized and normalized.

Synthetic share. +100 headwear and +100 underwear images generated with SD-Turbo (Section 3.2). They make up 200 / 1,245 = **16.1 %** of the augmented training set and **0 %** of validation and test, so every reported metric is computed on real images only.

Class	Train (real)	Val	Test	Synthetic (train)
bottomwear	210	45	45	0
footwear	210	45	45	0
headwear	205	43	45	100
topwear	210	45	45	0
underwear	210	45	45	100
Total	**1045**	**223**	**225**	**200**

Table 1. Dataset composition after the update.



Figure 1. Samples of the SYNTHETIC images (SD-Turbo). They are labeled in file names (synthetic_*.jpg) and figure titles.

3. SOTA Methods

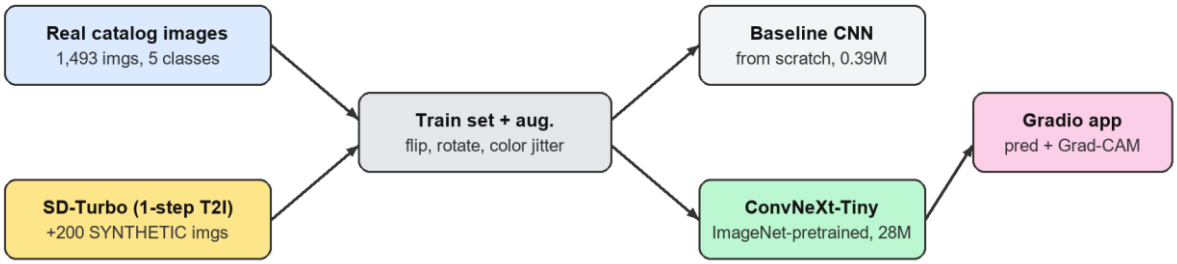


Figure 2. Overall pipeline: real and synthetic data → two classifiers → ablation → interactive app.

3.1 ConvNeXt-Tiny (transfer learning)

ConvNeXt [1] is a pure convolutional network that is "modernized" step by step toward the design of Vision Transformers. As the Neurohive CNN guide [5] summarizes, it borrows Transformer design choices and stays competitive with Transformers of the same size. The key changes over ResNet-50 are:

- **Patchify stem:** a 4×4 convolution with stride 4 instead of a 7×7 convolution plus max-pooling.
- **Stage ratio 3:3:9:3**, similar to Swin-T, with an inverted-bottleneck block: 7×7 depthwise conv → LayerNorm → 1×1 conv (4× expand) → GELU → 1×1 conv.
- **Large 7×7 depthwise kernels**, which mimic the large receptive field of self-attention windows.
- **LayerNorm instead of BatchNorm, GELU instead of ReLU**, fewer activations and normalizations, and separate downsampling layers.

We use the Tiny variant (27.8 M parameters, ImageNet-1k weights from timm), replace the head with a 5-way linear layer, and fine-tune all layers with AdamW (lr 1e-4, weight decay 1e-4, cosine schedule, 6 epochs, batch size 32). **Why it helps our topic:** the baseline confuses small, low-texture garments (underwear vs headwear and topwear). Pretrained features already encode fabric texture, garment shape and part layout, which about 1k training images cannot teach from scratch.

3.2 SD-Turbo synthetic data (Adversarial Diffusion Distillation)

SD-Turbo is Stable Diffusion 2.1 distilled with **Adversarial Diffusion Distillation (ADD)** [2]. A student network learns to produce an image in one denoising step. It is trained with (a) a score-distillation loss toward a frozen multi-step teacher and (b) an adversarial loss from a discriminator built on frozen ViT features. The result is a 512×512 image from a single forward pass, about 0.1 s on a T4.

We generate 100 images for each of the two weakest baseline classes from the template:

```
"e-commerce product photo of a {color} {item}, isolated on plain white
 background, centered, studio lighting, no person"
```

This uses 10 colors × 8 item types per class (e.g. baseball cap, beanie, fedora; boxer briefs, sports bra, thermal vest). Settings: 1 step, guidance 0, fixed seeds, resized to 256×256. **Why it helps:** it adds intra-class diversity (colors and sub-types) where the baseline recall is lowest, at almost zero cost and with no need to photograph or scrape new data.

4. Experiments

Setup. Google Colab, NVIDIA Tesla T4, PyTorch + timm. Loss: cross-entropy. The checkpoint with the best validation accuracy is evaluated on the test set. Latency is the pure forward pass at batch size 1 on the GPU, averaged over 50 runs after warm-up. The baseline CNN (A1-style) has 4 blocks of Conv3×3-BN-ReLU-MaxPool (32→256 channels), global average pooling, dropout 0.3 and an FC layer. It is trained for 15 epochs with AdamW at lr 1e-3.

4.1 Main results: baseline vs SOTA

Model	Train data	Accuracy	Macro-F1	Params	Latency	Train time
Baseline CNN	real	0.809	0.802	0.39 M	0.8 ms	89 s
ConvNeXt-Tiny	real	0.982	0.982	27.8 M	5.9 ms	161 s
Baseline CNN	real + syn	0.800	0.791	0.39 M	0.8 ms	115 s
ConvNeXt-Tiny	real + syn	0.987	0.987	27.8 M	6.8 ms	189 s

Table 2. Test-set results (225 real images).

Class	CNN	CNN + syn	ConvNeXt	ConvNeXt + syn
bottomwear	0.921	0.943	1.000	0.989
footwear	0.889	0.854	1.000	1.000
headwear	0.688	0.681	0.977	1.000
topwear	0.854	0.840	0.978	0.978
underwear	0.657	0.638	0.956	0.967

Table 3. Per-class F1 on the test set.

ConvNeXt removes almost all baseline errors: +17.3 pp accuracy and +0.18 macro-F1. The biggest gains are on the two hard classes, headwear (+0.29 F1) and underwear (+0.30 F1). The cost is 70× more parameters and about 7× higher latency, which is still far below real-time limits.

4.2 Training curves and confusion matrices

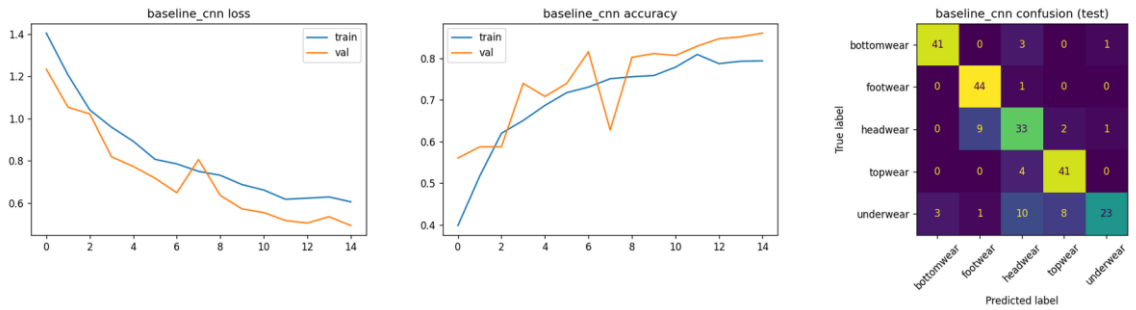


Figure 3. Baseline CNN: loss, accuracy, confusion matrix (test).

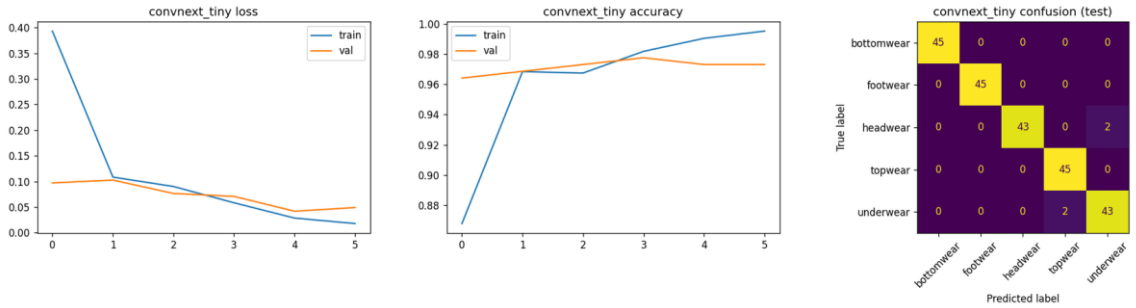


Figure 4. ConvNeXt-Tiny: loss, accuracy, confusion matrix (test).

The baseline converges slowly, and its validation accuracy oscillates between 0.63 and 0.86. Its dominant error is underwear predicted as headwear or topwear (only 23 / 45 correct), plus headwear predicted as footwear (9). ConvNeXt reaches 96 % validation accuracy after the first epoch. Its only test errors are 2 headwear → underwear and 2 underwear → topwear. A train/val gap appears after epoch 4, so more epochs would only overfit.

4.3 Ablation: with vs without synthetic data

Model	Real only (F1)	Real + synthetic (F1)	Δ Macro-F1	Δ Accuracy
Baseline CNN	0.802	0.791	−0.011	−0.9 pp
ConvNeXt-Tiny	0.982	0.987	+0.005	+0.45 pp

Table 4. Ablation of the SD-Turbo synthetic images.

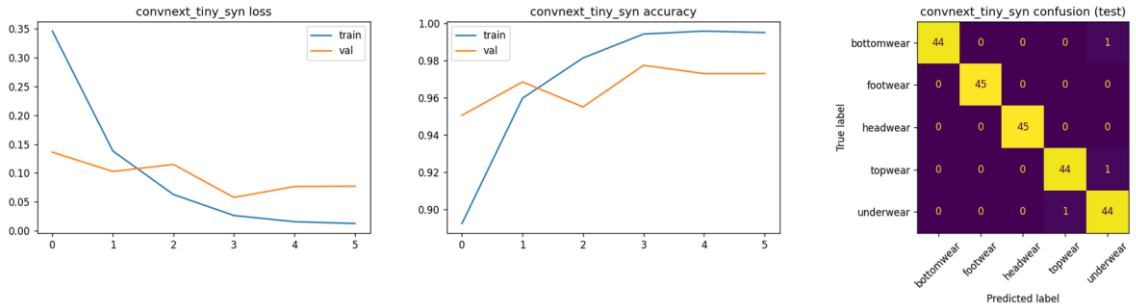


Figure 5. ConvNeXt-Tiny trained with synthetic data: headwear errors drop to 0.

Discussion. Synthetic data helps the pretrained ConvNeXt: headwear recall goes from 0.956 to 1.000, underwear F1 from 0.956 to 0.967, and the remaining errors are single images. It does not help the CNN trained from scratch. SD-Turbo images are high-resolution, smoothly shaded renders, while real catalog photos are small (60×80 px) and often show a person wearing the item. A network without a pretrained prior picks up generator-specific cues instead of garment shape, so the domain gap outweighs the added diversity. Also, several "underwear" generations look like athletic shorts, which overlap with bottomwear. With a single seed and 225 test images, the ConvNeXt gain equals **one** test image. The honest conclusion is "no harm, small gain on the targeted classes", not a significant improvement. Training time grows by about 18 %, and inference cost is unchanged (the 5.9 vs 6.8 ms difference is measurement noise).

5. Explainability (Grad-CAM)

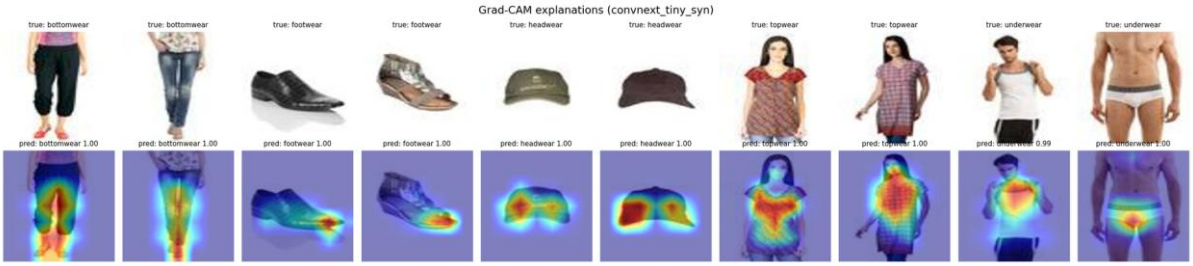


Figure 6. Grad-CAM on the last ConvNeXt stage, 2 test images per class (all correct, confidence 0.99–1.00).

Grad-CAM [3] weights the last-stage feature maps by the gradient of the predicted class score. The heatmaps show that the model attends to the garment rather than the white background: the leg split and trousers for bottomwear, the toe of the shoe for footwear, the whole cap for headwear, and the briefs and waistband for underwear. For **topwear**, however, the heat spreads to the model's **face and neck**. The classifier partly uses "a person's upper body is visible" as a cue for topwear, which is a shortcut learned from the catalog. Section 6 shows that this shortcut causes real errors.

6. Demo Application

The app is built with Gradio and runs in the Colab notebook (public share link). Flow: **upload or webcam → top-5 class probabilities → Grad-CAM overlay → latency (per request and running average) and environment** (GPU, torch and Python versions, model name). The interface states that part of the training data is synthetic and that uploads are not stored. End-to-end latency is **10–16 ms** per image on a Tesla T4 (torch 2.14, Python 3.13), including preprocessing and host-device transfer. A local version (app/app.py) runs on CPU or GPU.

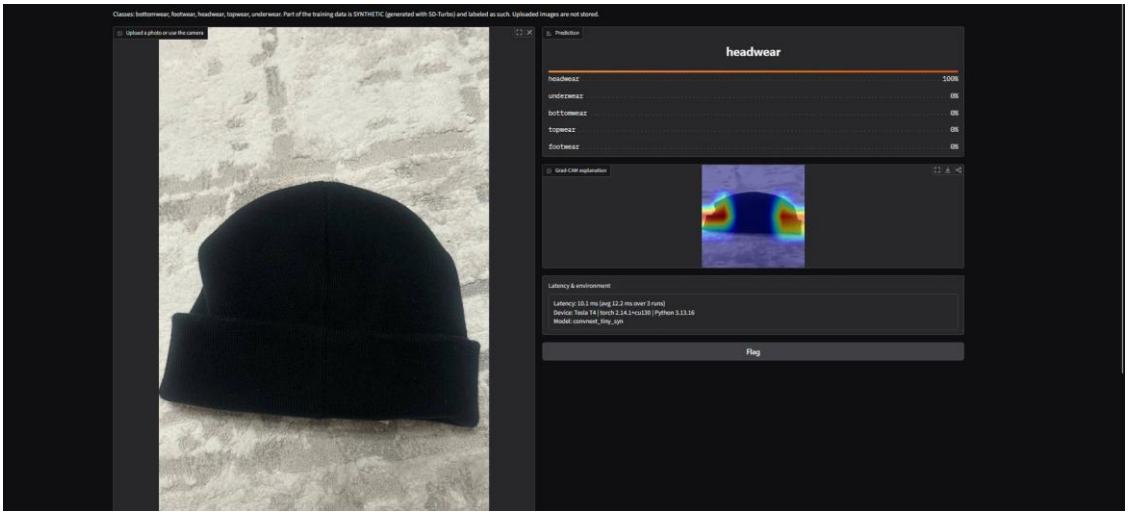


Figure 7. App on a real phone photo: knit beanie → headwear 100 %.

Real-world test with our own phone photos (garments lying flat on a carpet, no person):

Photo	Prediction	Correct?	Grad-CAM focus
Track pants	bottomwear 94 %	yes	crotch / leg split
Knit beanie	headwear 100 %	yes	rounded side contour
T-shirt (flat)	underwear 98 %	no (topwear)	small patch on the chest

Table 5. Out-of-distribution check in the app.

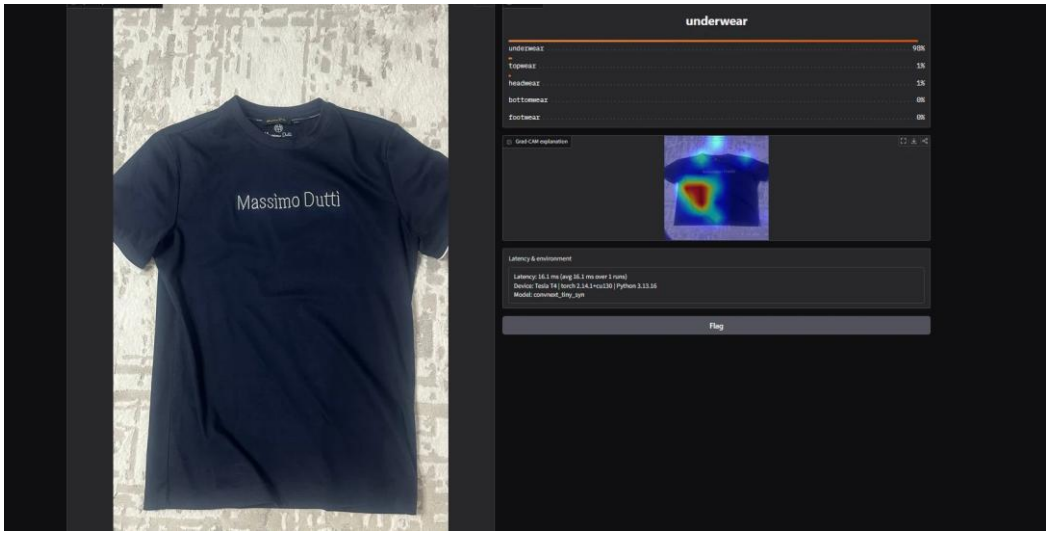


Figure 8. Failure case: a flat-lying T-shirt is classified as underwear.

The T-shirt failure confirms the shortcut seen in Section 5. In the catalog, almost every topwear image shows the garment **worn by a model**, while flat-lying cotton items without a person are mostly undershirts and vests from Innerwear. The model learned "flat cotton garment with no person → underwear" instead of the T-shirt's shape. The 98.7 % test accuracy therefore measures in-distribution performance only. Possible fixes: add flat-lay topwear photos, segment the garment (SAM-style mask) and remove the person and background before classification, and use stronger crop and background-replacement augmentation.

7. Ethics and Limitations

- **Data and consent.** The dataset is a public e-commerce catalog. Many images show professional models wearing the garments, including underwear, with visible faces. These are published retail photos used for non-commercial coursework, not images of private individuals. We do not redistribute the raw images (the notebook downloads them from the original source), and the app does not store uploads.
- **Synthetic media disclosure.** All generated images are labeled SYNTHETIC (file names, figure titles, app description) and excluded from validation and test to avoid inflated metrics.
- **Generated content risk.** Although prompts say "no person", SD-Turbo sometimes rendered partial bodies (legs or torso, never faces) for underwear prompts. The diffusers safety checker is disabled by default for this model. Samples were reviewed visually, and the generations are used only for training and are not exposed in the app.
- **Bias.** One retailer, mostly white studio backgrounds and a specific regional fashion distribution. Accuracy on street photos, worn clothing or other cultures' garments will be lower (see Table 5).
- **Limitations.** Small test set (225 images) and a single seed, so differences under about 1 pp are not significant. "Topwear" mixes shirts and jackets. Latency is measured on a cloud T4 only.

8. Reproducibility

1. Open the Colab notebook (link above) → Runtime → Change runtime type → **T4 GPU**.
2. Runtime → **Run all**. Cells: data download and split → baseline → ConvNeXt → SD-Turbo generation → ablation → Grad-CAM gallery and Gradio app. Total time is about 20 minutes.
3. Weights are saved to /content/{baseline_cnn, convnext_tiny, *_syn}.pt, and metrics and plots to /content/results. The last cell prints a public gradio.live link.
4. Local run: pip install -r requirements.txt; python app/app.py --weights convnext_tiny_syn.pt.

Environment: Python 3.13, torch 2.14 (CUDA 13), timm, diffusers, grad-cam, gradio. Seeds: data split 42, generation seeds 0–99 and 1000–1099. Code, notebook, figures and metrics (results/summary.csv) are in the GitHub repository.

9. Conclusion

Replacing the A1-style CNN with a pretrained ConvNeXt-Tiny is by far the most effective upgrade (+17 pp accuracy). One-step SD-Turbo synthetic data is cheap and gives a small further gain for the pretrained model on the targeted classes, but it does not help a model trained from scratch. Grad-CAM combined with a simple real-world test exposed a "person visible → topwear" shortcut that a 98.7 % test score hides. Garment segmentation before classification and flat-lay training images are the most promising next steps.

References

[1] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie, "A ConvNet for the 2020s," CVPR 2022.

[2] A. Sauer, D. Lorenz, A. Blattmann, R. Rombach, "Adversarial Diffusion Distillation," arXiv:2311.17042, 2023.

[3] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," ICCV 2017.

[4] P. Aggarwal, "Fashion Product Images (Small)," Kaggle; HF mirror: huggingface.co/datasets/ashraq/fashion-product-images-small.

[5] Neurohive, "Convolutional Neural Network (CNN): How It Works, With PyTorch" (ConvNeXt section), https://neurohive.io/en/guides/convolutional-neural-network/

[6] R. Wightman, "PyTorch Image Models (timm)," GitHub, 2019–2026.